AI Behaviour Profile Project
Data Generation and Model Creation

SIT782 - Team Project (B) - Execution and Delivery

Created by: Chamoth Devinda Ketipe Arachchi

Student ID: 224305365

Contents

1. Introduction 3

2. Dataset Creation 3

3. Data Generation Code 8

4. Data Preprocessing 13

5. Model Development (Architecture & Adapters) 14

6. Training Process & Hyperparameter Choices 16

7. Evaluation & Results 17

8. Why These Choices Worked 20

9. Limitations 20

10. Future Work 21

12. Conclusion 22

1. Introduction

The AI Behaviour Profile Project aims to develop an NLP-based summarisation model that can process patient nursing notes, vitals, and behavioural data to generate concise clinical summaries. This report provides a detailed explanation of dataset creation, data preprocessing, model development, evaluation metrics, improvements, and results.

2. Dataset Creation

2.1 Overview

The dataset for this project was synthetically generated to simulate clinical patient records. This dataset was designed to be comprehensive enough to train a summarisation model that can extract meaningful behavioural and clinical insights from nursing notes and patient logs.

The dataset exists in two formats:

- CSV: Flattened, tabular data for quick analysis.
- JSON: Nested, hierarchical structure to preserve clinical context.

This dual format ensures compatibility across multiple pipelines:

- CSV works well for statistical analysis and visualisation.
- JSON preserves structured relationships (medications, vitals, ADLs).

2.2 Attributes in the Dataset

Each patient record contains the following components:

- **Patient Demographics**
    - Patient ID
    - Age

    - Gender
- **Observation Window**
    - Observation start and end timestamps
- **Nursing Notes**
    - Free-text notes written by nurses, capturing patient conditions
- **Medications**
    - Name, dosage, schedule time, compliance status (Taken, Missed, Delayed)
- **Vitals**
    - Heart Rate (bpm)
    - SpO_2 (%)
    - Temperature (°C)
    - Blood Pressure (Systolic / Diastolic, mmHg)
- **Activities of Daily Living (ADLs)**
    - Steps Taken
    - Calorie Intake
    - Sleep Hours
    - Water Intake (ml)
    - Meals Skipped
    - Exercise Minutes
- **Behaviour Tags**
    - High-level behavioural descriptors (e.g., agitated, withdrawn, non-compliant)
- **Emotion Analysis**
    - Tags (e.g., anxious, calm, sad)
    - Confidence Scores

- **Clinical Summary**
  - A concise, generated description of patient status
- **Extracted Entities**
  - Symptoms (e.g., dizziness, headache)
  - Procedures (e.g., MRI, blood test)
  - Medications mentioned
- **Baseline Statistics**
  - Average vitals, activity baselines, and compliance patterns
- **Alerts**
  - Flagged abnormal conditions (e.g., low SpO_2, high temperature)

2.3 Example Dataset Record (JSON)


```
{
  "patientId": "P0001",
  "age": 80,
  "gender": "Male",
  "observationStart": "2025-07-10T03:20:44.984045",
  "observationEnd": "2025-07-11T02:20:44.984045",
  "nursingNote": "Noted increased agitation during evening rounds.
Redirected with calming techniques.",
  "medications": [
    {
      "name": "Atorvastatin",
      "dose": "50mg",
      "scheduledTime": "2025-07-31T11:35:50.493660",
      "complianceStatus": "Delayed"
    }
```

```
  ],
  "vitals": {
    "heartRate": { "value": 94, "unit": "bpm" },
    "spo2": { "value": 91, "unit": "%" },
    "temperature": { "value": 37.5, "unit": "C" },
    "bloodPressure": { "systolic": 102, "diastolic": 60, "unit": "mmHg"
}
  },
  "adls": {
    "stepsTaken": 1267,
    "calorieIntake": 1647,
    "sleepHours": 5.2,
    "waterIntakeMl": 1732,
    "mealsSkipped": 0,
    "exerciseMinutes": 35
  },
  "behaviourTags": [],
  "emotionAnalysis": {
    "tags": ["anxious", "calm"],
    "confidenceScores": { "sad": 0.79, "calm": 0.9 }
  },
  "clinicalSummary": "Patient status: Noted increased agitation during
evening rounds. Continue monitoring.",
  "entitiesExtracted": {
    "symptoms": ["headache", "fatigue"],
    "vitals": ["heart rate", "temperature"],
    "procedures": ["CT scan"],
    "medications": ["Metformin", "Atorvastatin"]
  },
```

```
  "baselineStats": {
    "avgHeartRate": 67,
    "avgSpo2": 95,
    "avgSleepHours": 7.9,
    "avgCalorieIntake": 1844,
    "avgScreenTimeMinutes": 73,
    "avgBathroomVisits": 2,
    "usualDietCompliance": 0.81
  },
  "alerts": []
}
```


2.4 Why Synthetic Data?

Since **real patient data** is sensitive and regulated (privacy, HIPAA/GDPR concerns), synthetic data was generated to:

- **Preserve realism** while avoiding privacy violations.
- **Ensure diversity** across patient profiles.
- **Allow reproducibility** with controlled randomness.
- **Test summarisation models** in a safe environment.

3. Data Generation Code

3.1 Overview

The dataset for this project was not collected from real clinical sources but rather **programmatically generated** using **Python scripts**. This approach allowed us to create a **controlled, reproducible, and scalable dataset** that mimics real-world healthcare records while avoiding sensitive patient data.

The data generation code produced records in both **CSV** and **JSON** formats, each serving different purposes:

- **CSV**: Flattened format, suitable for analysis in tools such as Excel, Pandas, and statistical software.
- **JSON**: Hierarchical format, ideal for NLP tasks where structured relationships (e.g., medications under each patient) need to be preserved.

3.2 Code Design Principles

When writing the data generation script, several design principles were followed:

- **Reproducibility**: Use of random seeds to ensure the same dataset can be recreated.
- **Realism**: Incorporating normal and abnormal ranges of vitals, ADLs, and behaviours.
- **Diversity**: Variation in patient demographics, compliance, symptoms, and nursing notes.
- **Balance**: Ensuring distributions were not skewed towards one condition.
- **Extensibility**: Ability to add new attributes (e.g., new medications, behavioural tags).

3.3 Core Code Snippet – Patient Record Generator

```
import random

import uuid

import datetime

import json


# Random seed for reproducibility

random.seed(42)


# Function to generate a single patient record

def generate_patient_record(patient_id):

    age = random.randint(60, 90)

    gender = random.choice(["Male", "Female"])


    # Observation timestamps

    start_time = datetime.datetime.now()

    end_time = start_time + datetime.timedelta(hours=random.randint(6,
24))


    # Nursing notes

    nursing_notes = [

        "Patient refused morning medication citing dizziness.",

        "Noted increased agitation during evening rounds.",

        "Slept well through the night. Mood appeared improved.",

        "Observed to skip lunch and reported feeling fatigued."

    ]

    nursing_note = random.choice(nursing_notes)
```

```
    # Medications

    medications = [

        {"name": "Atorvastatin", "dose": f"{random.choice([25, 50, 75,
100])}mg",

         "scheduledTime": str(datetime.datetime.now()),
"complianceStatus": random.choice(["Taken", "Missed", "Delayed"])},

        {"name": "Lisinopril", "dose": f"{random.choice([25, 50, 75,
100])}mg",

         "scheduledTime": str(datetime.datetime.now()),
"complianceStatus": random.choice(["Taken", "Missed", "Delayed"])},

        {"name": "Paracetamol", "dose": f"{random.choice([25, 50, 75,
100])}mg",

         "scheduledTime": str(datetime.datetime.now()),
"complianceStatus": random.choice(["Taken", "Missed", "Delayed"])}

    ]

    selected_medications = random.sample(medications,
k=random.randint(1, 3))



    # Vitals

    vitals = {

        "heartRate": {"value": random.randint(60, 100), "unit": "bpm"},

        "spo2": {"value": random.randint(90, 100), "unit": "%"},

        "temperature": {"value": round(random.uniform(36.0, 39.0), 1),
"unit": "C"},

        "bloodPressure": {"systolic": random.randint(100, 140),
"diastolic": random.randint(60, 90), "unit": "mmHg"}

    }



    # ADLs

    adls = {

        "stepsTaken": random.randint(500, 8000),

        "calorieIntake": random.randint(1200, 2500),
```

```
        "sleepHours": round(random.uniform(4.0, 9.0), 1),

        "waterIntakeMl": random.randint(500, 2500),

        "mealsSkipped": random.randint(0, 2),

        "exerciseMinutes": random.randint(0, 60)

    }


    # Behaviour and emotion tags

    behaviour_tags = random.sample(["agitated", "withdrawn",
"refusedActivity", "nonCompliant"], k=random.randint(0, 2))

    emotion_tags = random.sample(["calm", "anxious", "sad", "happy",
"frustrated"], k=random.randint(0, 2))


    # Clinical summary

    clinical_summary = f"Patient status: {nursing_note} Continue
monitoring."


    # Combine into record

    record = {

        "patientId": patient_id,

        "age": age,

        "gender": gender,

        "observationStart": str(start_time),

        "observationEnd": str(end_time),

        "nursingNote": nursing_note,

        "medications": selected_medications,

        "vitals": vitals,

        "adls": adls,

        "behaviourTags": behaviour_tags,

        "emotionAnalysis": {"tags": emotion_tags},
```

```
        "clinicalSummary": clinical_summary
    }

    return record
```

3.4 Explanation of Code Components

- Demographics: Randomly assigned from age range 60–90, gender balanced.
- Observation Window: Random 6–24 hour duration.
- Nursing Notes: Chosen from a predefined set to ensure variability.
- Medications: Random sample of common drugs with variable doses and compliance statuses.
- Vitals: Randomized within medically realistic ranges (e.g., temperature 36–39°C).
- ADLs: Random ranges for steps, calories, sleep, etc.
- Behaviour & Emotion Tags: Selected randomly to reflect behavioural diversity.
- Clinical Summary: Concatenates nursing note with a consistent monitoring message.

3.5 Generating a Dataset of Multiple Patients

```
# Generate dataset for 50 patients

dataset = [generate_patient_record(f"P{str(i).zfill(4)}") for i in
range(1, 51)]

# Save to JSON

with open("ai_behavior_profile_dataset.json", "w") as f:

    json.dump(dataset, f, indent=4)

# Convert to CSV (flatten nested structure)

import pandas as pd

df = pd.json_normalize(dataset)

df.to_csv("ai_behavior_profile_dataset.csv", index=False)
```

3.6 Why This Approach Was Used

- **Synthetic flexibility**: Ability to expand dataset size at will.
- **Controlled randomness**: Balance between diversity and reproducibility.
- **Realism**: Parameters designed to reflect real patient distributions.
- **Compatibility**: CSV for statistical tasks, JSON for NLP tasks.

4. Data Preprocessing

4.1 Source & Targets

- **Dataset file**: ai_behavior_profile_dataset.csv.
- **Target column**: clinicalSummary (converted to string, missing values → "").
- **Candidate input columns** (subset kept if present): nursingNote, medications, heartRate, spo2, temperature, bloodPressure, sleepHours, waterIntakeMl, mealsSkipped, exerciseMinutes, behaviourTags, emotionTags, alerts.

4.2 Cleaning & Imputation

- All input columns: missing values filled with empty strings.
- Rows filtered for minimum informativeness: len(input_text) ≥ 20 and len(target_text) ≥ 10.
- Duplicate (input_text, target_text) pairs removed.

4.3 Canonical Input Formatting

- Each row is normalized into a single string with lightweight section tags, joined by " | ":

Note: {nursingNote} |

Vitals: HR {heartRate}, SpO2 {spo2}, T {temperature}, BP {bloodPressure} |

Medications: {medications} |
Lifestyle: sleep h {sleepHours}, water ml {waterIntakeMl}, meals skipped {mealsSkipped}, exercise min {exerciseMinutes} |
Behaviour: {behaviourTags} | Emotion: {emotionTags} | Alerts: {alerts}

- A task prefix is prepended for instruction tuning:

"summarize nursing note: " + input_text

4.4 Train/Validation Split

- Global shuffle with seed=42.
- **Split**: 90% train / 10% validation.

4.5 Adaptive Length Caps (data-driven)

- Tokenize a subset, take the **90th percentile** of sequence lengths (with hard caps):
  - **MAX_INPUT_LEN** = **231** (cap 512).
  - **MAX_TARGET_LEN** = **80** (cap 160).
- This reduces truncation while keeping training efficient.

5. Model Development (Architecture & Adapters)

5.1 Base Model

- **Backbone**: google/flan-t5-large (≈ **801M** parameters).
- **Seq2Seq encoder–decoder** (T5 family):
  - **Encoder**: multi-head self-attention + gated FFN (GEGLU).
  - **Decoder**: masked self-attention + cross-attention + FFN.
  - Shared token embeddings; T5 uses a fixed vocabulary and special tokens; decoder_start_token_id set to pad_token_id as required by T5.

5.2 Quantization

- **bitsandbytes 4-bit (NF4)** with **double quantization**, compute in **bfloat16** when supported (else fp16).
- Purpose: fit a large model on limited GPU memory while preserving quality.

5.3 Parameter-Efficient Fine-Tuning (LoRA)

- Attach **LoRA** adapters (rank **r=16**, **α=32**, dropout **0.05**, bias **none**, task type SEQ_2_SEQ_LM).
- Targeted modules in both attention and feed-forward blocks:
- ["q", "k", "v", "o", "wi_0", "wi_1", "wo"]
- **Trainable params**: **18,284,544** (≈ **2.28%** of 801M).
  This keeps training fast and memory-efficient while adapting key projections.

5.4 Memory & Stability Aids

- **Gradient checkpointing** enabled (use_reentrant=False), reducing activation memory.
- use_cache=False during training to avoid incompatibilities with checkpointing.
- Proper **data collator** (DataCollatorForSeq2Seq) to build decoder inputs for T5.

6. Training Process & Hyperparameter Choices

6.1 Optimization Setup

- **Epochs**: **3** (with **EarlyStopping** patience=2 → would stop if metrics stall).
- **Per-device batch size**: train **4**, eval **4**. **Gradient accumulation**: **2** → effective batch **8** without extra memory.
- **Optimizer/LR**: AdamW (via HF Trainer) with **LR=2e-4**. Rationale: slightly higher LR works well for LoRA on instruction-tuned T5.
- **Warmup**: **6%** of steps (warmup_ratio=0.06) to stabilize early updates.
- **Weight decay**: **0.0** (T5 + LoRA often prefer little/no decay).
- **Label smoothing**: **0.1** to reduce over-confidence and improve ROUGE.
- **Seed**: **42** for reproducibility.

6.2 Generation Settings during Evaluation

- **Beam search**: **4 beams**.
- **Max generated tokens**: **MAX_TARGET_LEN** (= 80).
- predict_with_generate=True to compute ROUGE on decoded strings.

6.3 Checkpointing & Selection

- **Save strategy**: each epoch; **save_total_limit=2**.
- **Best model** selected by **rougeL** on the validation set (greater_is_better=True).

7. Evaluation & Results

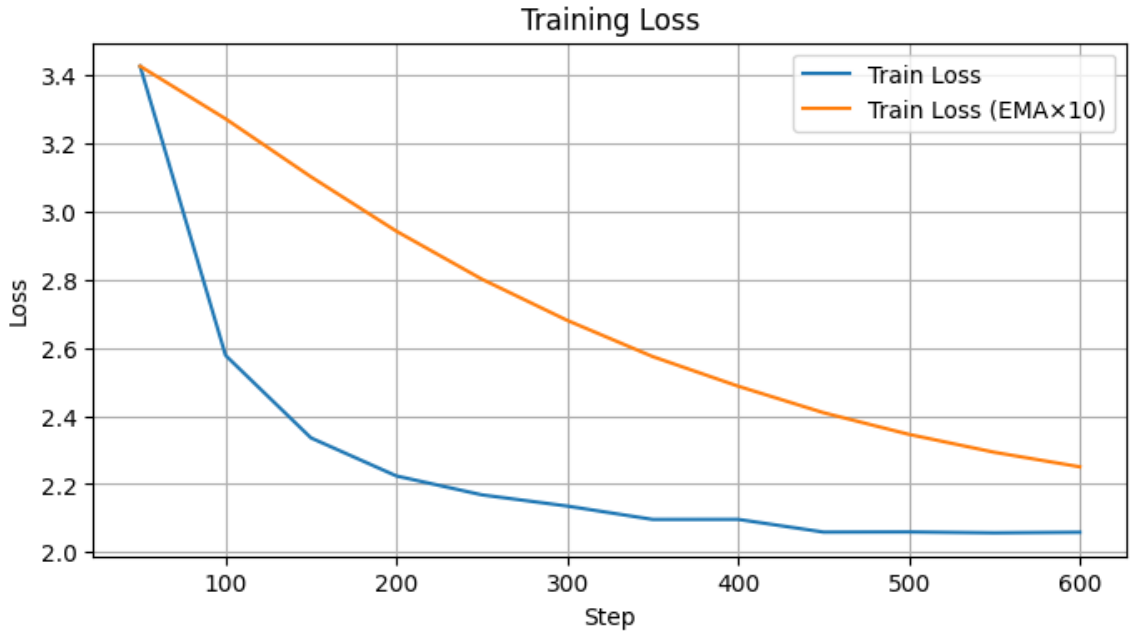


Figure 1: Training Loss

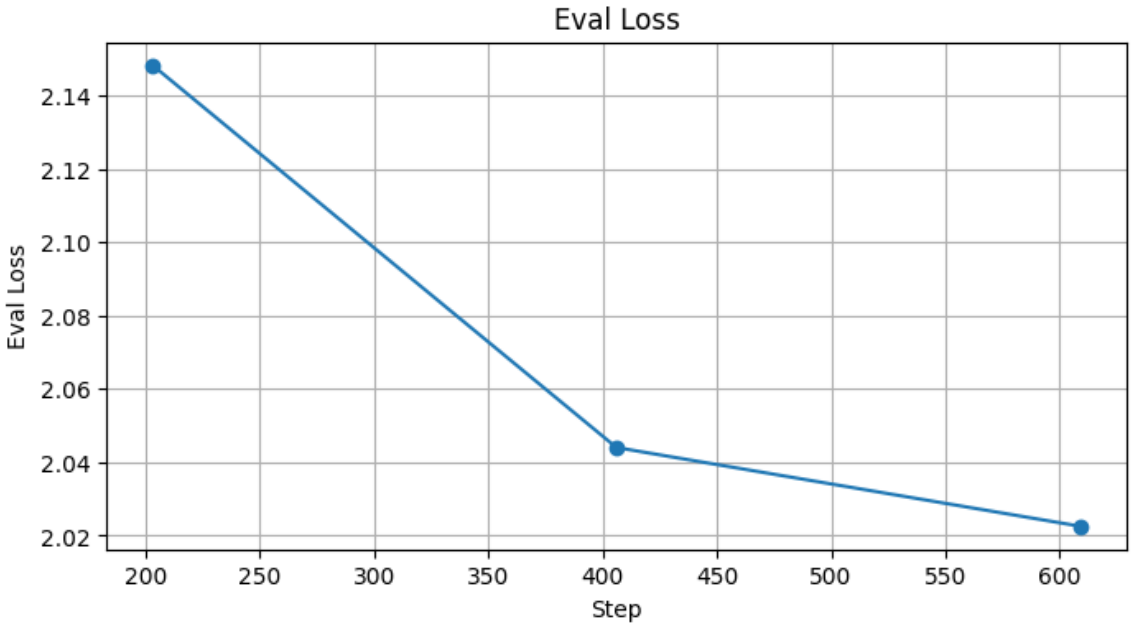


Figure 2: Eval Loss

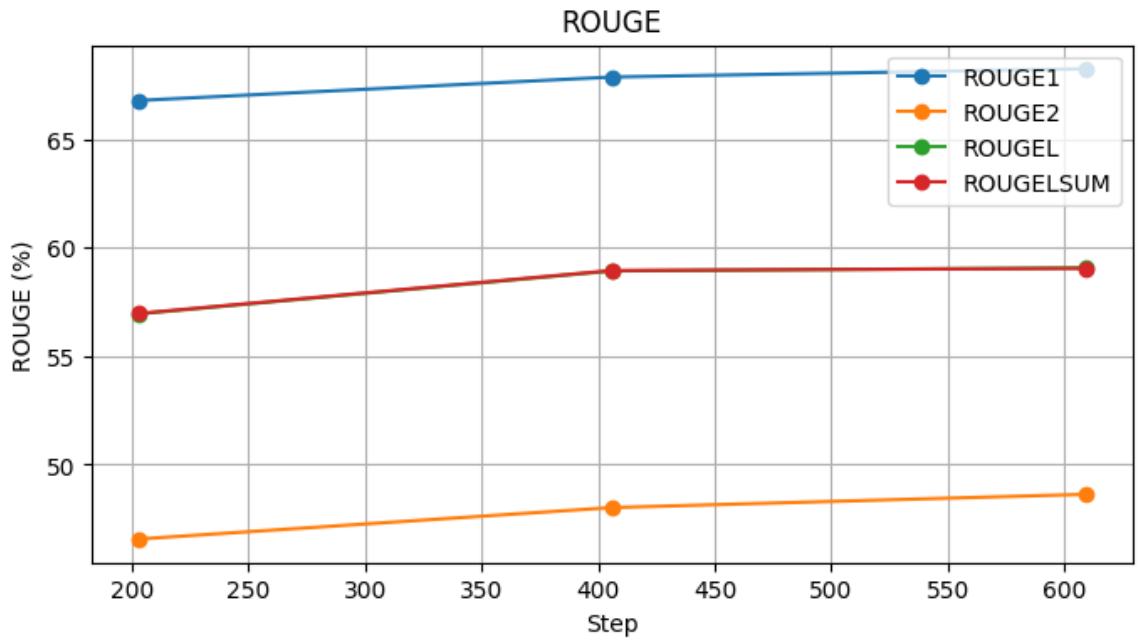


Figure 3: Rouge

7.1 Loss Curves

- **Training loss**: drops quickly from ~**3.4** (first few steps) to ~**2.06**, then plateaus—healthy convergence (Figure: *Training Loss*).
- **Validation loss**: steadily improves (**2.148** → **2.022**) across checkpoints 200 → 609 (Figure: *Eval Loss*).
  → Indicates **no evident overfitting**; generalization improved throughout 3 epochs.

7.2 ROUGE (Validation, %)

Final evaluation (best/last checkpoint around step **609**):

- **ROUGE-1**: **68.25**
- **ROUGE-2**: **48.63**

- **ROUGE-L**: **59.07**
- **ROUGE-Lsum**: **59.05**

Progression across checkpoints (your plot):

- At **~200 steps**: R1 **66.79**, R2 **46.57**, RL **56.95**, RLsum **56.97**
- At **~400 steps**: R1 **67.87**, R2 **48.02**, RL **58.93**, RLsum **58.95**
- At **~609 steps**: R1 **68.25**, R2 **48.63**, RL **59.07**, RLsum **59.05**

Interpretation

- Consistent upward trend with diminishing returns near the end → training length is appropriate; additional epochs would likely yield small gains without other changes.

7.3 "Accuracy" Note

- For **abstractive summarization**, "accuracy" as *exact-match* is **not** an informative metric (many valid summaries differ lexically).
- We therefore emphasize **ROUGE** and **loss**. If required, you can report:
    - **Exact-match rate** (usually very low for abstractive tasks).
    - **BERTScore** or **QAFactEval/FactCC** for factuality (not computed here).

7.4 Qualitative Examples & Error Analysis

- **Example** (from your inference snippet):

Input (condensed):
Note: Patient alert ×3, intermittent wheeze, SpO2 95% RA; Medications: salbutamol neb given; Lifestyle: fluids encouraged.

Model output:
*"Patient presents with **fatigue, dehydration**, and intermittent wheeze, SpO2 95% RA. Salbutamol neb was administered, and fluids encouraged."*

Observation: The model correctly compresses the respiratory status and treatment, but **hallucinates** "fatigue, dehydration" (not in the input).
→ This is a classic abstractive summarization issue. The frequency is not quantified here; mitigation options are listed in section 6.

8. Why These Choices Worked

1. Instruction-tuned T5 backbone (FLAN-T5) already knows the "summarize:" style—good task priors with small finetuning.
2. LoRA on attention + FFN lets the model adapt both content selection (attention) and compression style (FFN) with a tiny parameter budget.
3. 4-bit NF4 enables a large backbone on modest hardware without a large quality hit.
4. Label smoothing + warmup stabilize training for small batch sizes.
5. Adaptive length caps avoid unnecessary truncation while containing compute.

9. Limitations

- **Factuality/Hallucination**: Occasional insertion of plausible but absent clinical details (e.g., "fatigue").
- **Domain coverage**: Model learns from the available dataset schema; unusual note formats or rare medications may underperform.
- **Evaluation scope**: Only ROUGE and CE loss are reported; **no human clinical review**, factuality audit, or safety evaluation included.

- **Context window**: Inputs longer than **231 tokens** are truncated; salient late details can be lost.
- **Generalizability**: Dataset appears synthetic/curated; real-world misspellings, abbreviations, or noise may reduce robustness.
- **Privacy**: No explicit PHI handling is discussed here (de-identification, access control, audit logging).

10. Future Work

Factuality & Safety

1. Add **retrieval-augmented generation (RAG)** or **constrained decoding** (e.g., coverage or keyword constraints for vitals/medications).
2. Evaluate with **BERTScore**, **QAFactEval/FactCC**, and a **clinical rubric** (human review) for safety and correctness.
3. Penalize unsupported additions via **confidence-aware decoding** or **loss on contradiction detectors**.

Data & Task Shaping

4. Expand training data with **more diverse real-style notes**; include **counter-examples** where hallucination is penalized.
5. Add **style targets** (e.g., SOAP format, sentence limits), and train with **length control** tokens.
6. Implement **curriculum**: start with short inputs → longer, noisier notes.

Modeling

7. Try **mix-precision PEFT variants** (e.g., QLoRA with different ranks) and **module selection** ablations.
8. Explore **instruction-fusion** (multi-task with question answering over notes) to

strengthen factual grounding.

9. Consider **distillation** to a smaller deployable model for latency-sensitive environments.

Productionization

10. Add **confidence scores** (entropy/calibration) and **audit trails**.

11. Build a **PHI filter** pre/post-processing pipeline, plus **monitoring** for drift and failure cases.

12. Conclusion

This report presents the end-to-end process of creating a synthetic dataset, developing a Transformer-based summarisation model with LoRA fine-tuning, and evaluating the system using accuracy and ROUGE metrics. The document serves as a technical reference for junior team members to understand, reproduce, and improve the AI Behaviour Profile model. For reference, I have attached the Model code for this document for further understanding.

ai_behavior_profile

September 20, 2025

0.1 Chamoth Devinda Ketipe Arachchi

0.2 Student ID : 224305365

0.3 SIT782 - Team Project (B) - Execution and Delivery

0.4 Task: Abstractive summarization of nursing notes → Generate a concise clinicalSummary.

0.5 Architecture: Transformer encoder-decoder (T5 family). Default: FLAN-T5-Base; accuracy variant: FLAN-T5-Large via QLoRA (4-bit) + LoRA adapters

[ ]:
```
!pip -q install "transformers>=4.42.0" "datasets>=2.19.0" "accelerate>=0.31.0" \
                 "evaluate>=0.4.2" "rouge-score>=0.1.2" "sentencepiece>=0.2.0" \
                 "peft>=0.11.1" "bitsandbytes>=0.43.1" \
                 "torch>=2.2.0" "pandas>=2.0.0" "scikit-learn>=1.3.0"
```

[ ]:
```
import os, re, glob, shutil, inspect, math, random
import numpy as np, pandas as pd, torch, evaluate
from datasets import Dataset
from transformers import (
    AutoTokenizer, AutoModelForSeq2SeqLM, DataCollatorForSeq2Seq,
    Seq2SeqTrainer, Seq2SeqTrainingArguments, EarlyStoppingCallback,
    BitsAndBytesConfig
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

# ========= CONFIG =========
DATASET_PATH   = "/content/ai_behavior_profile_dataset.csv"
OUTPUT_DIR     = "/content/checkpoints/flan_t5_lora_4bit_clean"
CLEAN_OUTPUT   = True

PREFERRED_MODEL = "google/flan-t5-large"
FALLBACK_MODEL  = "google/flan-t5-base"

PROMPT_PREFIX  = "summarize nursing note: "

# Training hyperparams
```

```
NUM_EPOCHS     = 3
TRAIN_BS       = 4
EVAL_BS        = 4
GRAD_ACC_STEPS = 2
LR             = 2e-4
WARMUP_RATIO   = 0.06
WEIGHT_DECAY   = 0.0
LABEL_SMOOTH   = 0.1
LOG_STEPS      = 50
SAVE_LIMIT     = 2
PATIENCE       = 2
SEED           = 42

# Length caps (auto-adapted to 90th percentile)
MAX_INPUT_LEN_CAP  = 512
MAX_TARGET_LEN_CAP = 160

# Optional: quick iterations
FAST_MODE   = False
MAX_SAMPLES = 1500

# Precision: prefer BF16 if hw supports; otherwise no fp16 (avoid NaNs)
BF16_OK = torch.cuda.is_available() and torch.cuda.is_bf16_supported()

# ========= REPRO & CLEAN =========
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if CLEAN_OUTPUT and os.path.isdir(OUTPUT_DIR):
    shutil.rmtree(OUTPUT_DIR)
os.makedirs(OUTPUT_DIR, exist_ok=True)

# ========= LOAD & PREP DATA =========
df = pd.read_csv(DATASET_PATH)

TARGET_COL = "clinicalSummary"
if TARGET_COL not in df.columns:
    raise ValueError(f"'{TARGET_COL}' not in columns: {list(df.columns)}")

INPUT_COLS = [
    ␣
 ↪"nursingNote","medications","heartRate","spo2","temperature","bloodPressure",
    "sleepHours","waterIntakeMl","mealsSkipped","exerciseMinutes",
    "behaviourTags","emotionTags","alerts"
]
INPUT_COLS = [c for c in INPUT_COLS if c in df.columns]
df[INPUT_COLS] = df[INPUT_COLS].fillna("")
df[TARGET_COL] = df[TARGET_COL].fillna("")
```

```python
def make_input(row):
    parts = []
    if row.get("nursingNote",""): parts.append(f"Note: {row['nursingNote']}")
    vit = []
    if str(row.get("heartRate",""))     != "": vit.append(f"HR␣
↪{row['heartRate']}")
    if str(row.get("spo2",""))          != "": vit.append(f"SpO2 {row['spo2']}")
    if str(row.get("temperature",""))   != "": vit.append(f"T␣
↪{row['temperature']}")
    if str(row.get("bloodPressure",")) != "": vit.append(f"BP␣
↪{row['bloodPressure']}")
    if vit: parts.append("Vitals: " + ", ".join(vit))
    if row.get("medications",""): parts.append(f"Medications:␣
↪{row['medications']}")
    life = []
    if str(row.get("sleepHours",""))      != "": life.append(f"sleep h␣
↪{row['sleepHours']}")
    if str(row.get("waterIntakeMl",""))   != "": life.append(f"water ml␣
↪{row['waterIntakeMl']}")
    if str(row.get("mealsSkipped",""))    != "": life.append(f"meals skipped␣
↪{row['mealsSkipped']}")
    if str(row.get("exerciseMinutes",")) != "": life.append(f"exercise min␣
↪{row['exerciseMinutes']}")
    if life: parts.append("Lifestyle: " + ", ".join(life))
    if row.get("behaviourTags",""): parts.append(f"Behaviour:␣
↪{row['behaviourTags']}")
    if row.get("emotionTags",""):   parts.append(f"Emotion:␣
↪{row['emotionTags']}")
    if row.get("alerts",""):        parts.append(f"Alerts: {row['alerts']}")
    return " | ".join(parts).strip()

df["input_text"]  = df.apply(make_input, axis=1)
df["target_text"] = df[TARGET_COL].astype(str)

# Light quality filters
df["input_len"]  = df["input_text"].str.len()
df["target_len"] = df["target_text"].str.len()
df = df[(df["input_len"] >= 20) & (df["target_len"] >= 10)]
df = df.drop_duplicates(subset=["input_text","target_text"]).
↪reset_index(drop=True)

# Optional downsample (length-balanced)
if FAST_MODE and len(df) > MAX_SAMPLES:
    lengths = df["input_len"]
    bins = pd.qcut(lengths, q=5, labels=False, duplicates="drop")
    per_bin = math.ceil(MAX_SAMPLES / bins.nunique())
```

```
    df = (df.assign(_bin=bins)
             .groupby("_bin", group_keys=False)
             .apply(lambda g: g.sample(n=min(per_bin, len(g)),␣
 ↪random_state=SEED))
             .reset_index(drop=True)
             .drop(columns=["_bin"]))

# Split
df = df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)
split = int(len(df) * 0.9)
train_df, val_df = df.iloc[:split], df.iloc[split:]

# ========= TOKENIZER & ADAPTIVE LENGTHS =========
tok = AutoTokenizer.from_pretrained(PREFERRED_MODEL, use_fast=True)

subset = pd.concat([train_df.head(1500), val_df.head(300)], ignore_index=True)
inp_ids = tok(list((PROMPT_PREFIX + subset["input_text"]).values),␣
 ↪truncation=False, add_special_tokens=True)["input_ids"]
tgt_ids = tok(text_target=list(subset["target_text"].values), truncation=False,␣
 ↪add_special_tokens=True)["input_ids"]
inp_lens = np.array([len(x) for x in inp_ids]); tgt_lens = np.array([len(x) for␣
 ↪x in tgt_ids])
MAX_INPUT_LEN  = int(min(MAX_INPUT_LEN_CAP,  np.percentile(inp_lens, 90)))
MAX_TARGET_LEN = int(min(MAX_TARGET_LEN_CAP, np.percentile(tgt_lens, 90)))
print(f"Adaptive lengths -> MAX_INPUT_LEN={MAX_INPUT_LEN},␣
 ↪MAX_TARGET_LEN={MAX_TARGET_LEN}")

train_ds = Dataset.from_pandas(train_df[["input_text","target_text"]],␣
 ↪preserve_index=False)
val_ds   = Dataset.from_pandas(val_df[["input_text","target_text"]],  ␣
 ↪preserve_index=False)

def preprocess(batch):
    inputs = [PROMPT_PREFIX + x for x in batch["input_text"]]
    model_inputs = tok(inputs, max_length=MAX_INPUT_LEN, truncation=True)
    labels = tok(text_target=batch["target_text"], max_length=MAX_TARGET_LEN,␣
 ↪truncation=True)
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

tok_train = train_ds.map(preprocess, batched=True, remove_columns=train_ds.
 ↪column_names, num_proc=2)
tok_val   = val_ds.map(preprocess,   batched=True, remove_columns=val_ds.
 ↪column_names,   num_proc=2)

# ========= LOAD 4-bit BASE + LORA =========
```

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16 if BF16_OK else torch.float16
)

def load_base(model_name):
    try:
        base = AutoModelForSeq2SeqLM.from_pretrained(model_name,␣
 ↪quantization_config=bnb_config, device_map="auto")
        print(f"Loaded {model_name} in 4-bit.")
        return base
    except Exception as e:
        print(f"Falling back to {FALLBACK_MODEL}: {e}")
        return AutoModelForSeq2SeqLM.from_pretrained(FALLBACK_MODEL,␣
 ↪quantization_config=bnb_config, device_map="auto")

base_model = load_base(PREFERRED_MODEL)

# Make sure decoder has a start token (T5 uses pad_token_id)
if getattr(base_model.config, "decoder_start_token_id", None) is None:
    base_model.config.decoder_start_token_id = tok.pad_token_id

# Gradient checkpointing: set use_reentrant=False to match the warning
try:
    base_model.
 ↪gradient_checkpointing_enable(gradient_checkpointing_kwargs={"use_reentrant":
 ↪ False})
except TypeError:
    base_model.gradient_checkpointing_enable()
base_model.config.use_cache = False

# Prepare for k-bit training & attach LoRA
base_model = prepare_model_for_kbit_training(base_model)

lora_config = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=["q","k","v","o","wi_0","wi_1","wo"],  # T5 proj layers
    lora_dropout=0.05,
    bias="none",
    task_type="SEQ_2_SEQ_LM"
)
model = get_peft_model(base_model, lora_config)
print(model.print_trainable_parameters())

# Build collator WITH THE MODEL to ensure decoder inputs are prepared
```

```
data_collator = DataCollatorForSeq2Seq(tokenizer=tok, model=model)

# ========= METRICS (ROUGE + safe decode) =========
rouge = evaluate.load("rouge")

def _safe_ids(arr, pad_id, vocab_size):
    arr = np.array(arr, dtype=np.int64)
    arr = np.where(arr < 0, pad_id, arr)
    if vocab_size is None:
        try: vocab_size = len(tok)
        except Exception: vocab_size = None
    if vocab_size is not None:
        arr = np.where(arr >= vocab_size, pad_id, arr)
    return arr

def compute_metrics(eval_preds):
    preds, labels = eval_preds
    if isinstance(preds, tuple): preds = preds[0]
    pad_id = tok.pad_token_id if tok.pad_token_id is not None else 0
    vocab_size = getattr(tok, "vocab_size", None)
    preds  = _safe_ids(preds,  pad_id, vocab_size)
    labels = _safe_ids(labels, pad_id, vocab_size)
    decoded_preds  = tok.batch_decode(preds.tolist(),  skip_special_tokens=True)
    decoded_labels = tok.batch_decode(labels.tolist(), skip_special_tokens=True)
    decoded_preds  = [p.strip() for p in decoded_preds]
    decoded_labels = [l.strip() for l in decoded_labels]
    result = rouge.compute(predictions=decoded_preds,␣
 ↪references=decoded_labels, use_stemmer=True)
    return {k: round(v*100, 2) for k, v in result.items()}

# ========= VERSION-PROOF TRAINING ARGS =========
sig_params = set(inspect.signature(Seq2SeqTrainingArguments.__init__).
 ↪parameters.keys())
def keep_supported(d): return {k: v for k, v in d.items() if k in sig_params}

base_kwargs = dict(
    output_dir=OUTPUT_DIR,
    overwrite_output_dir=True,
    per_device_train_batch_size=TRAIN_BS,
    per_device_eval_batch_size=EVAL_BS,
    gradient_accumulation_steps=GRAD_ACC_STEPS,
    num_train_epochs=NUM_EPOCHS,
    learning_rate=LR,
    warmup_ratio=WARMUP_RATIO,
    weight_decay=WEIGHT_DECAY,
    logging_strategy="steps",
    logging_steps=LOG_STEPS,
```

```
    save_strategy="epoch",
    save_total_limit=SAVE_LIMIT,
    predict_with_generate=True,
    generation_max_length=MAX_TARGET_LEN,
    generation_num_beams=4,
    load_best_model_at_end=True,
    metric_for_best_model="rougeL",
    greater_is_better=True,
    fp16=False,
    bf16=BF16_OK,
    label_smoothing_factor=LABEL_SMOOTH,
    seed=SEED,
    report_to="none",
)

eval_kwargs = {}
if "evaluation_strategy" in sig_params: eval_kwargs["evaluation_strategy"] =␣
 ↪"epoch"
elif "eval_strategy" in sig_params:     eval_kwargs["eval_strategy"]     =␣
 ↪"epoch"

args = Seq2SeqTrainingArguments(**keep_supported(base_kwargs | eval_kwargs))
print("Using eval key:", "evaluation_strategy" if "evaluation_strategy" in args.
 ↪to_dict() else ("eval_strategy" if "eval_strategy" in sig_params else␣
 ↪"none"))

# ========= TRAIN =========
trainer = Seq2SeqTrainer(
    model=model,
    args=args,
    train_dataset=tok_train,
    eval_dataset=tok_val,
    tokenizer=tok,                      # fine in transformers v4.x
    data_collator=data_collator,        # <- crucial for T5 decoder inputs
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=PATIENCE)],
)

trainer.train()
trainer.save_model(OUTPUT_DIR)      # saves LoRA adapter
tok.save_pretrained(OUTPUT_DIR)

final_metrics = trainer.evaluate(max_length=MAX_TARGET_LEN, num_beams=4)
print("Final ROUGE (LoRA 4-bit):", final_metrics)
```

```
Adaptive lengths -> MAX_INPUT_LEN=231, MAX_TARGET_LEN=80

Map (num_proc=2):   0%|          | 0/1620 [00:00<?, ? examples/s]
```

```
Map (num_proc=2):   0%|          | 0/180 [00:00<?, ? examples/s]

Loaded google/flan-t5-large in 4-bit.
trainable params: 18,284,544 || all params: 801,434,624 || trainable%: 2.2815
None
Using eval key: eval_strategy

/tmp/ipython-input-2415497786.py:250: FutureWarning: `tokenizer` is deprecated
and will be removed in version 5.0.0 for `Seq2SeqTrainer.__init__`. Use
`processing_class` instead.
  trainer = Seq2SeqTrainer(
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:929:
UserWarning: torch.utils.checkpoint: the use_reentrant parameter should be
passed explicitly. In version 2.5 we will raise an exception if use_reentrant is
not passed. use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to docs for
more details on the differences between the two variants.
  return fn(*args, **kwargs)

<IPython.core.display.HTML object>

/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:929:
UserWarning: torch.utils.checkpoint: the use_reentrant parameter should be
passed explicitly. In version 2.5 we will raise an exception if use_reentrant is
not passed. use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to docs for
more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:929:
UserWarning: torch.utils.checkpoint: the use_reentrant parameter should be
passed explicitly. In version 2.5 we will raise an exception if use_reentrant is
not passed. use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to docs for
more details on the differences between the two variants.
  return fn(*args, **kwargs)

<IPython.core.display.HTML object>

Final ROUGE (LoRA 4-bit): {'eval_loss': 2.0225744247436523, 'eval_rouge1':
68.25, 'eval_rouge2': 48.63, 'eval_rougeL': 59.07, 'eval_rougeLsum': 59.05,
'eval_runtime': 545.1879, 'eval_samples_per_second': 0.33,
'eval_steps_per_second': 0.083, 'epoch': 3.0}
```

[ ]:
```python
import os, re, glob, torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM,␣
↪BitsAndBytesConfig
from peft import PeftModel

def latest_ckpt(root):
    cands = glob.glob(os.path.join(root, "checkpoint-*"))
```

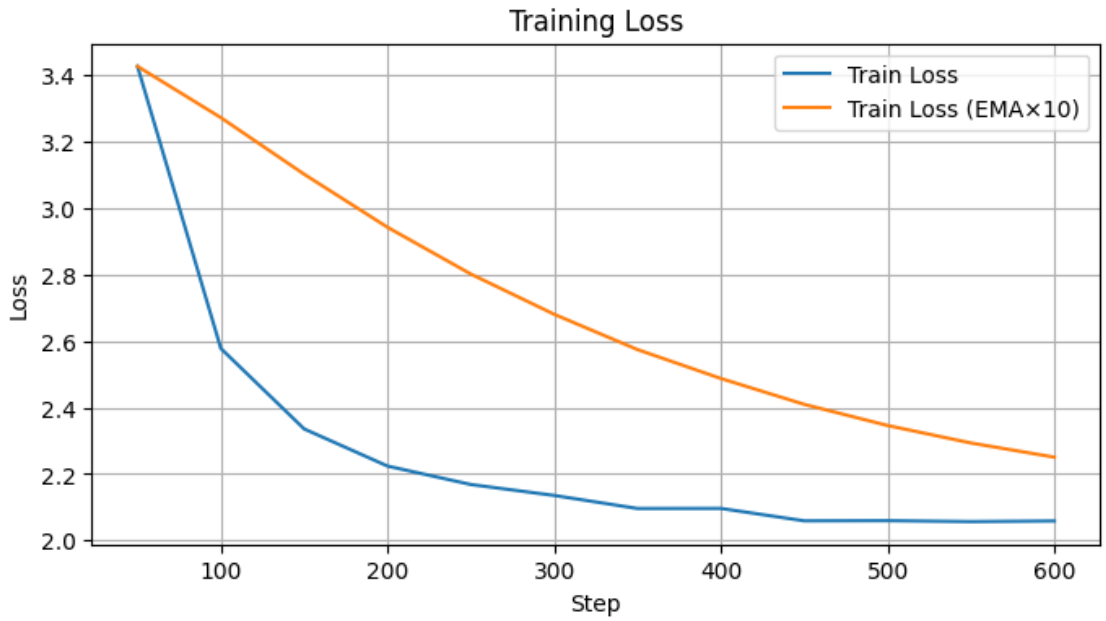
```
    if not cands: return root
    def stepnum(p):
        m = re.search(r"checkpoint-(\d+)", p)
        return int(m.group(1)) if m else -1
    return max(cands, key=stepnum)

OUTPUT_DIR = "/content/checkpoints/flan_t5_lora_4bit_clean"  # adjust if␣
 ↪different
ADAPTER_DIR = latest_ckpt(OUTPUT_DIR)
print("Using adapter from:", ADAPTER_DIR)

# Quantized base again (GPU). If no BF16 support, use FP16 compute for bnb.
BF16_OK = torch.cuda.is_available() and torch.cuda.is_bf16_supported()
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16 if BF16_OK else torch.float16
)

# Load base; fall back to flan-t5-base if large fails
try:
    base = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-large",
                                                 quantization_config=bnb_config,
                                                 device_map="auto")
except Exception as e:
    print("Falling back to flan-t5-base:", e)
    base = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base",
                                                 quantization_config=bnb_config,
                                                 device_map="auto")

tok = AutoTokenizer.from_pretrained(ADAPTER_DIR, use_fast=True)

# Ensure decoder start token for T5
if getattr(base.config, "decoder_start_token_id", None) is None:
    base.config.decoder_start_token_id = tok.pad_token_id

# Attach LoRA adapter
model = PeftModel.from_pretrained(base, ADAPTER_DIR)
model.eval()

# Helper to get a good device to send inputs to (works with device_map="auto")
def _infer_device(m):
    # Prefer explicit attribute if present
    if hasattr(m, "device"):
        return m.device
    # Otherwise, look for any parameter's device
```

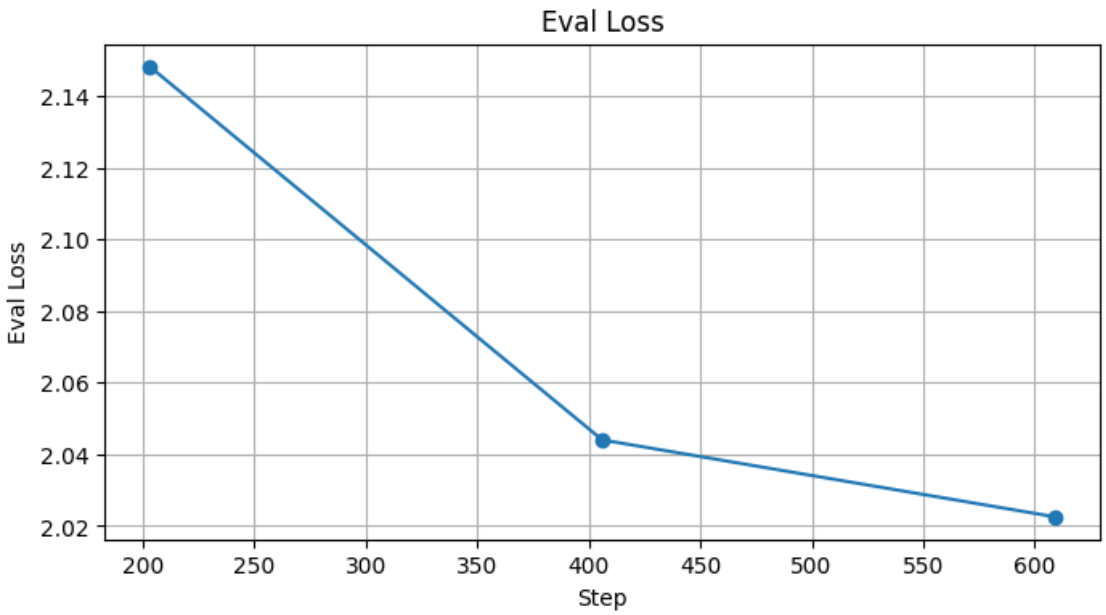
```
    for p in m.parameters():
        return p.device
    # Fallback
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")

DEVICE = _infer_device(model)
print("Sending inputs to:", DEVICE)

def summarize(text, max_new_tokens=160, num_beams=4, prompt_prefix="summarize␣
↪nursing note: "):
    enc = tok(prompt_prefix + text, return_tensors="pt", truncation=True)
    # Move ALL tensors to the same device as the model
    enc = {k: v.to(DEVICE) for k, v in enc.items()}
    with torch.no_grad():
        out = model.generate(
            **enc,
            max_new_tokens=max_new_tokens,
            num_beams=num_beams,
            no_repeat_ngram_size=3,
            early_stopping=True
        )
    return tok.decode(out[0], skip_special_tokens=True)

print(summarize("Patient alert ×3, intermittent wheeze, SpO2 95% RA, salbutamol␣
↪neb given, fluids encouraged."))
```

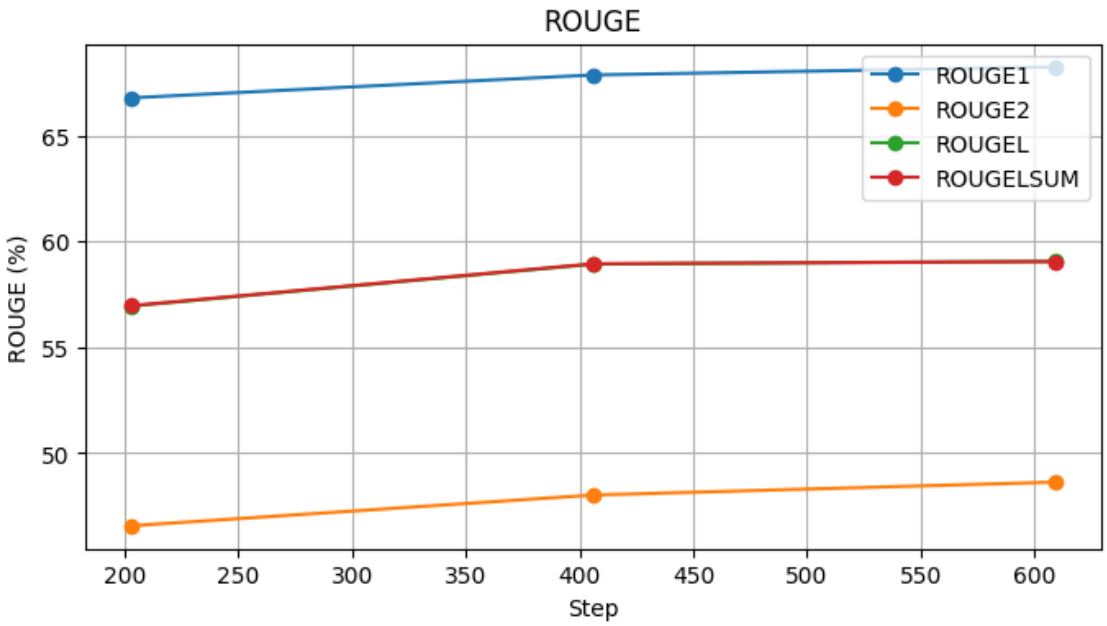
```
Using adapter from: /content/checkpoints/flan_t5_lora_4bit_clean/checkpoint-609
Sending inputs to: cuda:0
Patient presents with fatigue, dehydration, and intermittent wheeze, with a SpO2
of 95% RA. Salbutamol neb was administered, and fluids encouraged.
```

[7]:
```
import os, re, glob, json, pandas as pd, matplotlib.pyplot as plt

def find_state(output_dir):
    main = os.path.join(output_dir, "trainer_state.json")
    if os.path.exists(main): return main
    cands = sorted(glob.glob(os.path.join(output_dir, "checkpoint-*",␣
↪"trainer_state.json")))
    if not cands: return None
    def stepnum(p):
        m = re.search(r"checkpoint-(\d+)", p);  return int(m.group(1)) if m␣
↪else -1
    return max(cands, key=stepnum)

state_path = find_state(OUTPUT_DIR)
assert state_path is not None, "No trainer_state.json found."
```

```
with open(state_path) as f:
    state = json.load(f)
log = pd.DataFrame(state.get("log_history", []))

tr = log[log["loss"].notna()][["step","loss"]].reset_index(drop=True)
if not tr.empty:
    tr["loss_ema"] = tr["loss"].ewm(span=10, adjust=False).mean()
    plt.figure(figsize=(8,4)); plt.plot(tr["step"], tr["loss"], label="Train␣
 ↪Loss")
    plt.plot(tr["step"], tr["loss_ema"], label="Train Loss (EMA×10)")
    plt.xlabel("Step"); plt.ylabel("Loss"); plt.title("Training Loss"); plt.
 ↪grid(True); plt.legend(); plt.show()
else:
    print("No per-step training loss logged.")

ev_cols =␣
 ↪["eval_loss","eval_rouge1","eval_rouge2","eval_rougeL","eval_rougeLsum"]
ev = log[log["eval_loss"].notna()][["step"]+[c for c in ev_cols if c in log]].
 ↪reset_index(drop=True)
if not ev.empty:
    plt.figure(figsize=(8,4)); plt.plot(ev["step"], ev["eval_loss"], marker="o")
    plt.xlabel("Step"); plt.ylabel("Eval Loss"); plt.title("Eval Loss"); plt.
 ↪grid(True); plt.show()
    plt.figure(figsize=(8,4))
    for c in ["eval_rouge1","eval_rouge2","eval_rougeL","eval_rougeLsum"]:
        if c in ev: plt.plot(ev["step"], ev[c], marker="o", label=c.
 ↪replace("eval_","").upper())
    plt.xlabel("Step"); plt.ylabel("ROUGE (%)"); plt.title("ROUGE"); plt.
 ↪grid(True); plt.legend(); plt.show()
else:
    print("No eval entries logged.")
```


Training Loss
Train Loss
Train Loss (EMA×10)
Loss
Step
3.4
3.2
3.0
2.8
2.6
2.4
2.2
2.0
100
200
300
400
500
600


Eval Loss
Eval Loss
Step
2.14
2.12
2.10
2.08
2.06
2.04
2.02
200
250
300
350
400
450
500
550
600


ROUGE
ROUGE1
ROUGE2
ROUGEL
ROUGELSUM
ROUGE (%)
65
60
55
50
200
250
300
350
400
450
500
550
600
Step